Fontify — Setup & Operations Guide

Important behavior notes & database requirements for the typography utility app and Chrome extension

Next.js (App Router) · Supabase · Tailwind CSS · Chrome Extension (MV3)

This guide collects every **required manual step** and **important behavior caveat** surfaced while building the project — the things that code alone cannot configure. Follow the database setup first, then review the behavior notes for each module.

Part 1 — Database Requirements (Supabase)

The app uses Supabase for Google authentication and for persisting user data (saved font pairings and type-scale presets). The following must be configured in your Supabase project dashboard — they are not created automatically by the app.

> ■ **Required before saving features work**
>
> The **Font Pairing Engine** and **Type Scale Calculator** both read/write Supabase tables. If the migrations below are not run, the Save / Load buttons will silently return empty results or error on insert. The user must also be **logged in** — the server actions return early when there is no authenticated user.

1.1 — Environment variables

Create `.env.local` at the project root with your real Supabase credentials (placeholders are currently used so the build can run):

```
NEXT_PUBLIC_SUPABASE_URL=https://your-project-ref.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=your-anon-public-key

# Optional — Font Pairing Engine (free Google Fonts API)
NEXT_PUBLIC_GOOGLE_FONTS_KEY=your-google-fonts-api-key
```

1.2 — Enable Google authentication

- **Supabase** → **Authentication** → **Providers** → **Google**: enable it and paste your Google OAuth **Client ID** and **Client Secret**.
- **Google Cloud Console** → **OAuth credentials**: add Supabase's callback as an authorized redirect URI: `https://<project-ref>.supabase.co/auth/v1/callback`
- **Supabase** → **Authentication** → **URL Configuration**: add your site URLs to *Redirect URLs*, e.g. `http://localhost:3000/**` and your production domain.

1.3 — Run the table migrations

In **Supabase** → **SQL Editor**, paste and run each migration file from the project's `supabase/migrations/` folder. Both tables use Row Level Security so users can only access their own rows.

Migration A — `0001_saved_pairings.sql` (Font Pairing Engine)

```
create table if not exists public.saved_pairings (
  id           uuid primary key default gen_random_uuid(),
  user_id      uuid not null references auth.users (id) on delete cascade,
  tag          text not null,
  heading_font text not null,
  body_font    text not null,
  created_at   timestamptz not null default now()
);
alter table public.saved_pairings enable row level security;
-- + SELECT / INSERT / DELETE policies on auth.uid() = user_id
```

Migration B — `0002_scale_presets.sql` (Type Scale Calculator)

```
create table if not exists public.scale_presets (
  id         uuid primary key default gen_random_uuid(),
  user_id    uuid not null references auth.users (id) on delete cascade,
  name       text not null,
  base       numeric not null,
  ratio      numeric not null,
  min_vw     integer not null,
  max_vw     integer not null,
  created_at timestamptz not null default now()
);
alter table public.scale_presets enable row level security;
-- + SELECT / INSERT / DELETE policies on auth.uid() = user_id
```

> **■ Row Level Security (RLS) summary**
>
> Each table has three policies — **select**, **insert**, and **delete** — all gated on `auth.uid() = user_id`. There is intentionally no UPDATE policy. The full policy SQL is included in each migration file.

Part 2 — Important Behavior Notes

These are behavior caveats that affect how the app and extension run. They do not block the build, but they change what you should expect at runtime.

2.1 — Next.js version specifics

- **Async cookies():** In Next.js 15+/16, `cookies()` is async, so the server Supabase client is an `async` function — always call it as `await createClient()`.
- **middleware → proxy:** Next.js 16 renamed the `middleware.ts` convention to `proxy.ts`. The project uses `proxy.ts` (exported function `proxy()`) which calls `updateSession` to refresh the auth token on each request.

2.2 — Font Pairing Engine

- **Why not next/font:** `next/font` is build-time only and cannot load fonts chosen at runtime by a picker. The tool uses **manual <link> injection** into `document.head` instead.
- **API key is public:** The Google Fonts key is `NEXT_PUBLIC_` (exposed client-side). Restrict it by **HTTP referrer** in Google Cloud Console to prevent abuse.
- **Graceful fallback:** If the key is missing or the request fails, the tool loads a curated offline list (~30 fonts) and shows an amber badge — it never dead-ends.

2.3 — Variable Font Inspector

- **100% client-side:** Fonts are parsed in the browser with `opentype.js` — nothing is uploaded.
- **Blob @font-face in preview:** The live preview injects an `@font-face` via an object URL. This works fully in a real browser; in a sandboxed in-app preview, Blob URLs / font loading may be restricted (metadata, axes, and CSS output still render).

2.4 — In-app file preview sandbox

- Workspace file previews run in a sandboxed iframe with **no network access** — external stylesheets, scripts, fonts, and images will not load. Downloaded files work fully in a real browser.

> **■ Chrome Extension — the key UX caveat**
>
> A Chrome **popup closes the moment you click back onto the webpage** to hover an element. With the current design (content script → `chrome.runtime.sendMessage` → popup), you must keep the popup open to watch results update live. This is fine for a learning build but not ideal for everyday use.
>
> **Recommended fixes:** (1) store the latest reading in `chrome.storage` and show the last hovered element's styles when the popup reopens; or (2) render an on-page overlay/tooltip drawn by `content.js` instead of sending data to the popup.
>
> **Restricted pages:** Injection is blocked on `chrome://`, `about:`, and the Chrome Web Store — the popup guards against these.

Part 3 — Step-by-Step Setup

1 Install dependencies

From the project root, install all packages:

```
npm install
```

2 Configure environment variables

Copy the example file and fill in real values:

```
cp .env.local.example .env.local
# then edit .env.local
```

3 Set up Google OAuth

- Enable Google provider in Supabase (Client ID + Secret).
- Add the Supabase callback URL in Google Cloud Console.
- Add localhost + production URLs to Supabase Redirect URLs.

4 Run database migrations

In Supabase → SQL Editor, run both files in order:

- `supabase/migrations/0001_saved_pairings.sql`
- `supabase/migrations/0002_scale_presets.sql`

5 Run the app

Start the dev server and sign in with Google:

```
npm run dev   # http://localhost:3000
```

6 Verify the build (optional)

```
npx tsc --noEmit
npm run build
```

7 Load the Chrome extension

- Open `chrome://extensions` and enable **Developer mode**.
- Click **Load unpacked** and select the `extension/` folder.
- Open a normal webpage, click the icon → **Inspect Font**, then hover elements (keep the popup open).

Generated for the Fontify project · Keep this guide with your repo for onboarding and deployment.